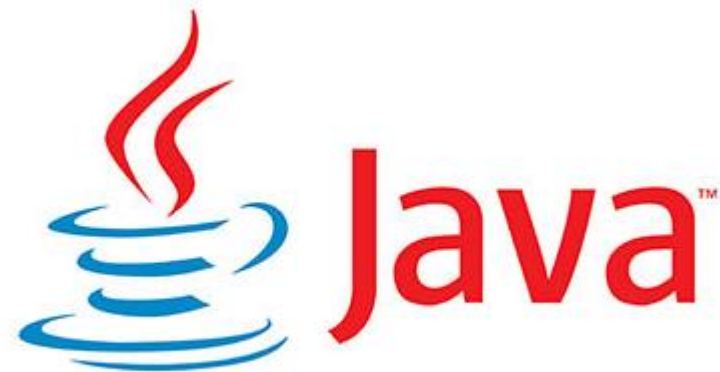




알기 쉽게 해설한 자바 10th Edition

1th week

JAVA 기초 핵심 문법 복습

Intro

강의 계획

강의 계획

- 강의 목표

- JAVA 언어 기반 애플리케이션 설계 및 구현 등 S/W 개발 능력 함양을 목표로 한다.

- 핵심 내용

- JAVA GUI, 스윙과 이벤트 처리, 컬렉션 프레임워크, JDBC 연동 응용 프로그램 구현

- 수업 평가

- 출결 20%, 태도 20%, 중간 20%(실기), 기말 40%(프로젝트)

- 수업 교재

- 알기 쉽게 해설한 자바(10th Edition), 김충석 저, 이한미디어

학습목표

- JAVA 애플리케이션 개발 환경 구축하기
- 객체지향프로그래밍의 개념과 특징 이해하기
 - (1) 클래스 정의
 - (2) 객체 생성
 - (3) 메시지 호출

* 캡슐화, 상속, 다형성 → 접근한정자, 인터페이스, 추상클래스
- JAVA 언어를 이용하여 간단한 콘솔 프로그램 구현해보기

01

핵심 문법 복습하기

개발환경구축

자바 개발환경 구축

■ JDK 설치

(<https://www.oracle.com/java/technologies/downloads/#java8-windows>)

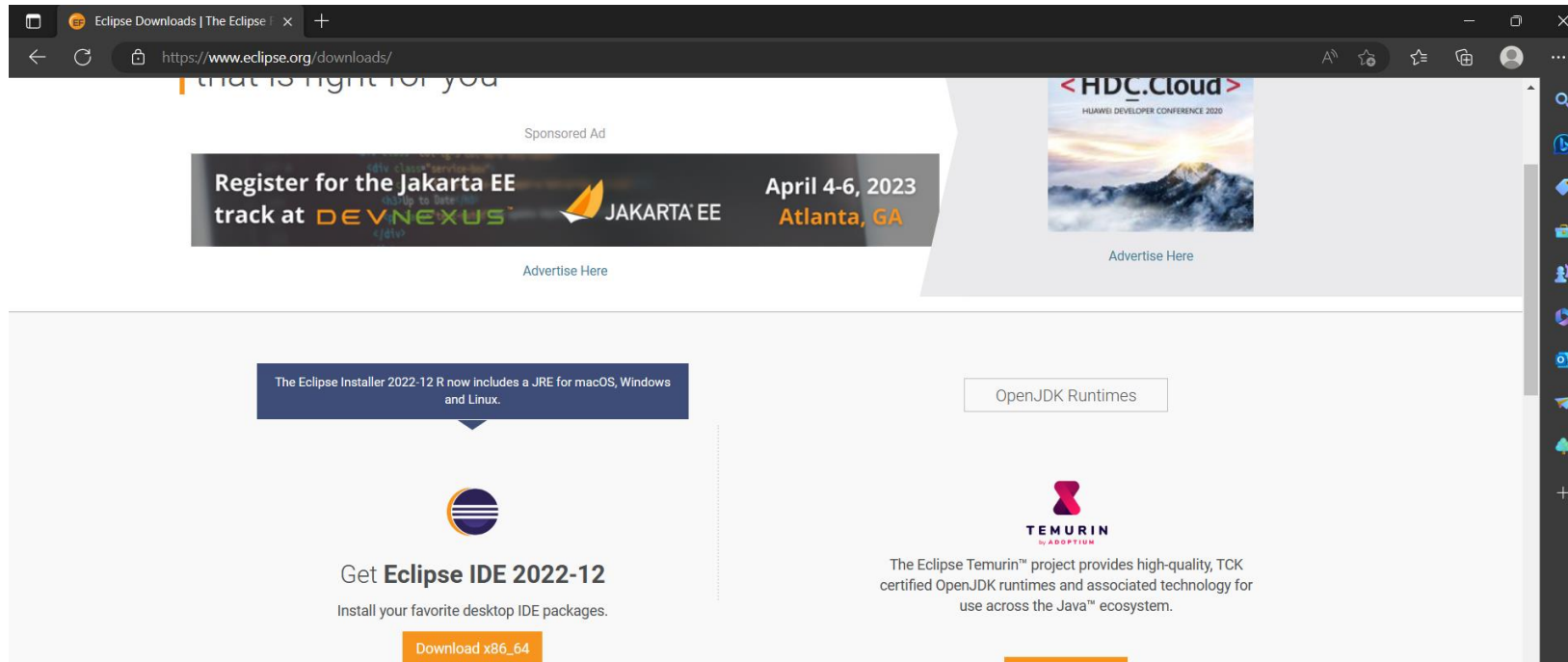
- JDK(Java Development Kit) : 자바 프로그램 개발에 필요한 라이브러리와 플랫폼 포함
- JRE(Java Runtime Environment) : 미리 만들어진 라이브러리를 묶어놓은 환경(API)
- JVM(Java Virtual Machine) : 자바 애플리케이션 실행 환경(JRE에 포함)

■ IDE 설치

- IDE(Integrated Development Environment, 통합 개발 환경)
- Eclipse, IntelliJ 등 → Eclipse 설치(<https://www.eclipse.org/downloads/>)

자바 개발환경 구축

<https://www.eclipse.org/downloads/>



All downloads are provided under the terms and conditions of the [Eclipse Foundation Software User Agreement](#) unless otherwise specified.

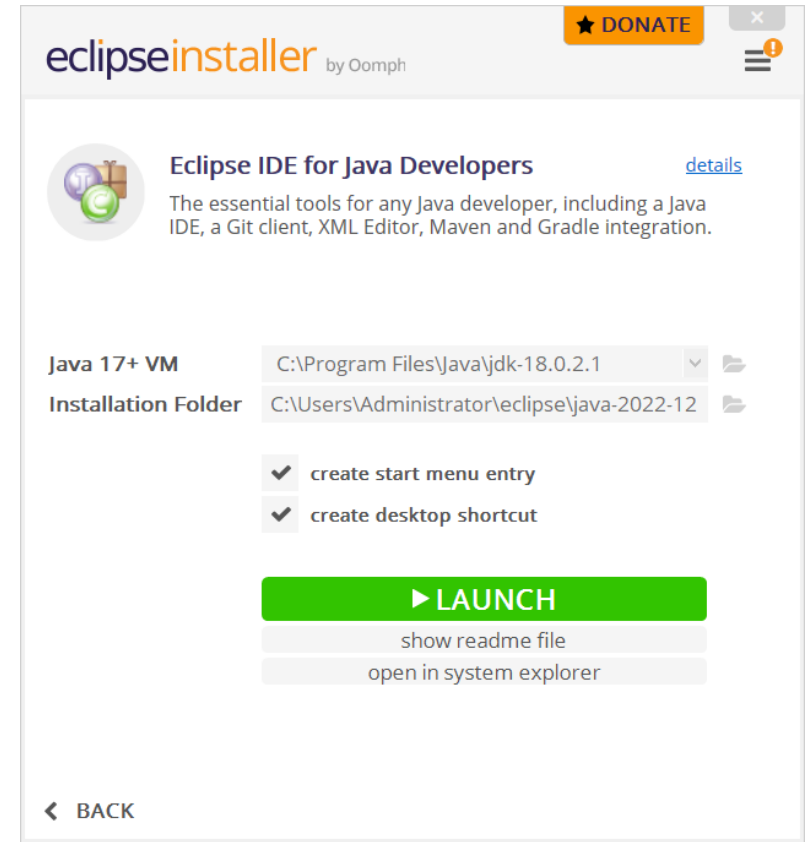
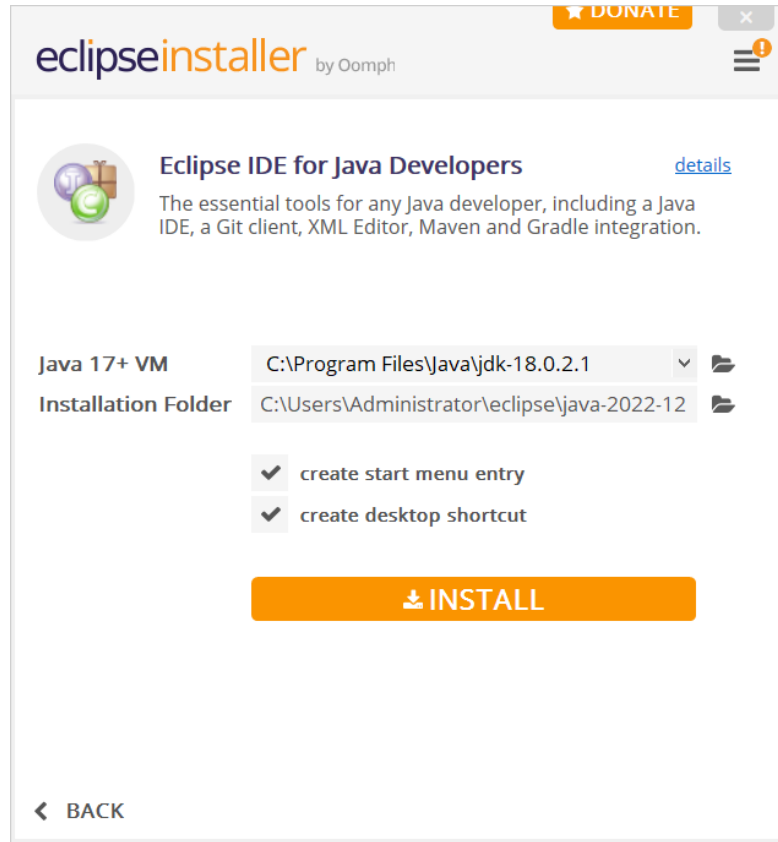
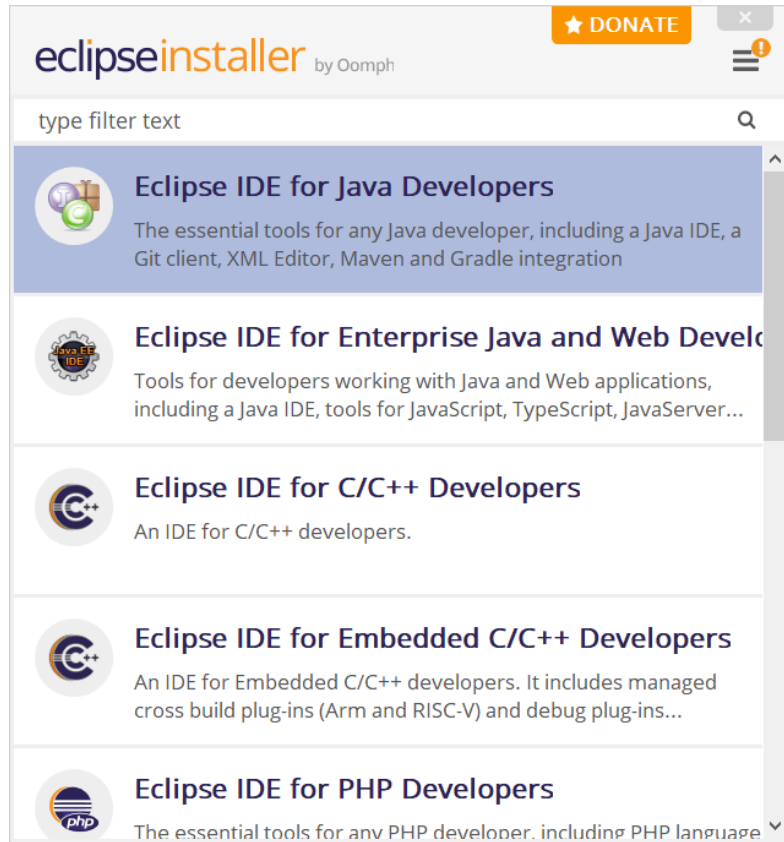
[Download](#)

Download from: Japan - Yamagata University ([https](https://www.yamagata-u.ac.jp/~eclipse/))

File: [eclipse-inst-jre-win64.exe](#) SHA-512

[>> Select Another Mirror](#)

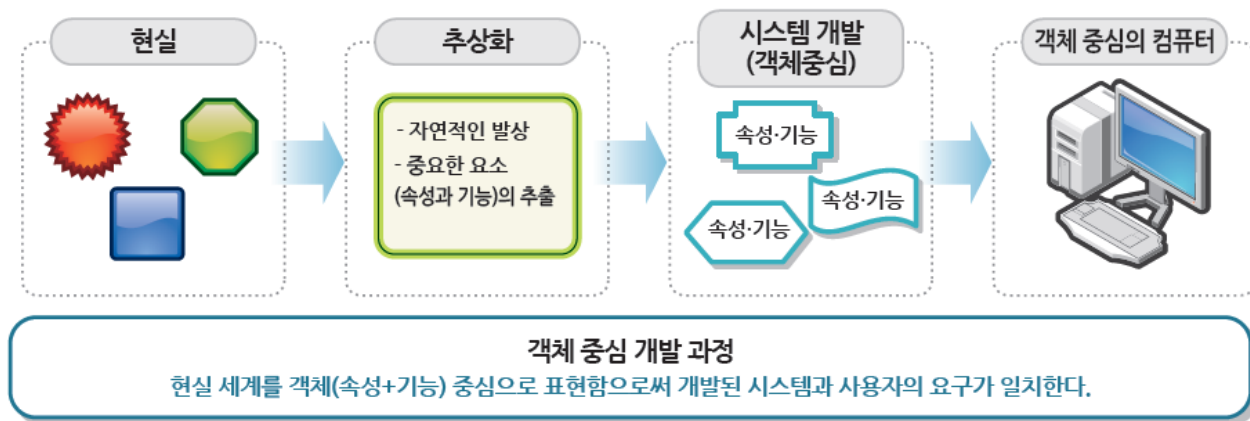
자바 개발환경 구축



객체지향 개념

객체지향 개념

- **객체지향**이론? 컴퓨터를 통해 **실세계와 같은 환경을 흉내내기** 위해 발전한 이론
 - 실세계 사물 = **속성 + 기능**
- 객체지향의 **장점**
 - 문제를 쉽고 자연스럽게 프로그래밍 가능
 - **쉬운 프로그램 개발**로 생산성 향상
 - 프로그램 모듈 **재사용** 가능



클래스와 객체

- **클래스**란? 여러 개의 **객체**를 생성하기 위한 **템플릿**(틀, 형판) (예) 학생
- **객체**란? **속성+기능**으로 구성, **클래스로부터 생성** (예) 홍길동, 성춘향
- **객체지향프로그래밍 3단계**? **클래스 설계**, **객체 생성**, **메시지 전송**(메소드 호출)

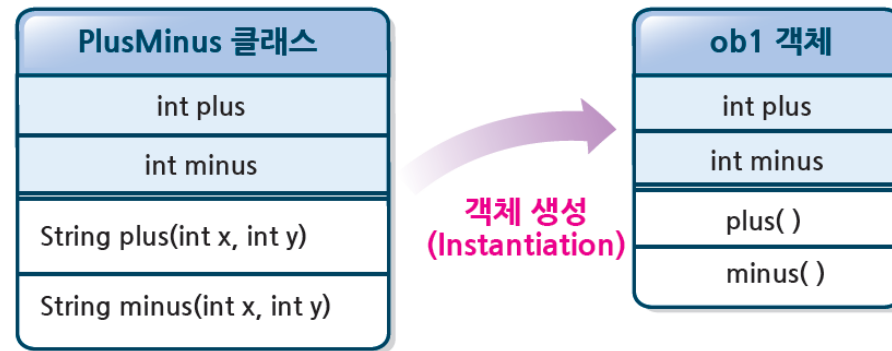
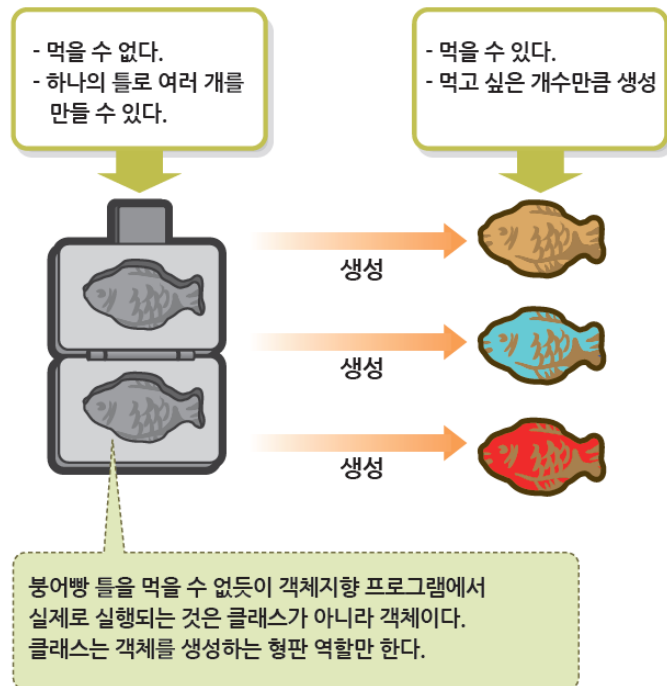


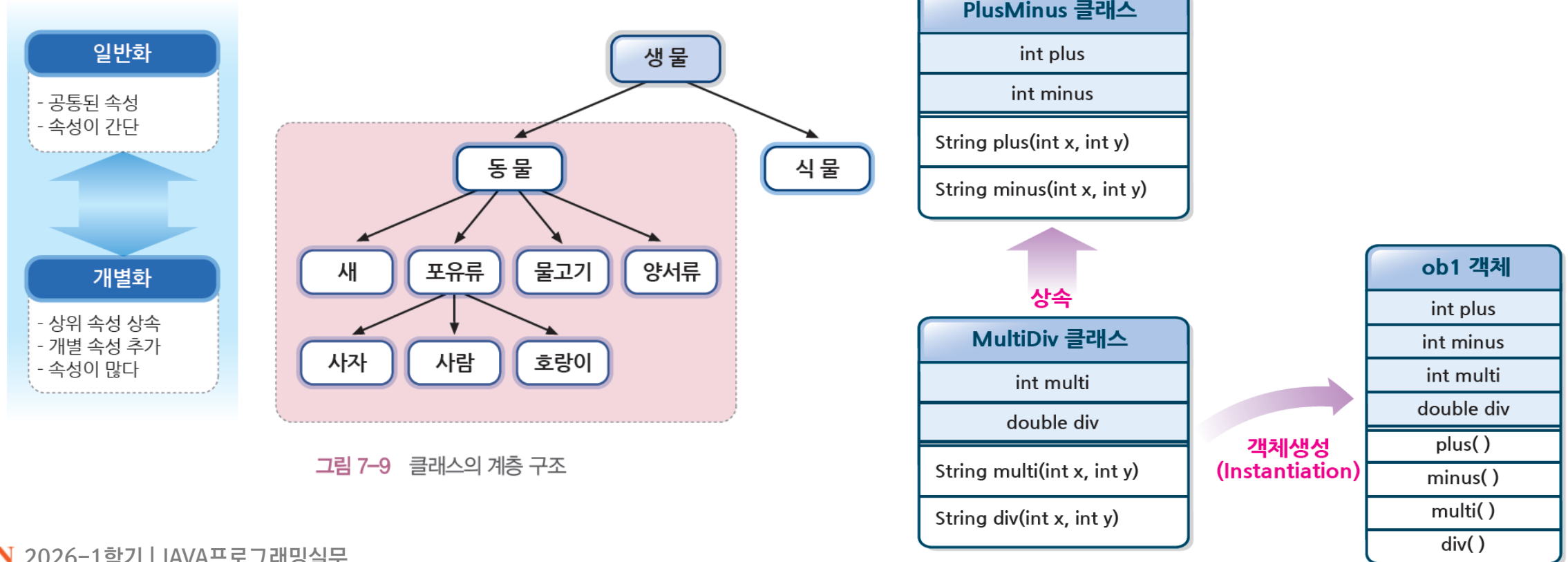
그림 7-7 PlusMinus 클래스로부터 ob1 객체의 생성

**클래스(PlusMinus)로부터 객체(ob1) 생성
이때, 객체를 인스턴스라고도 함**

객체지향 특징

상속(Inheritance)

- **상속?** 기존 구현한 클래스를 그대로 복사, 기능을 추가하여 **확장**하는 개념(**재사용**)
- 자바는 **다중 상속 허용 불가**, 모든 클래스는 최상위 클래스 **Object**의 하위 클래스



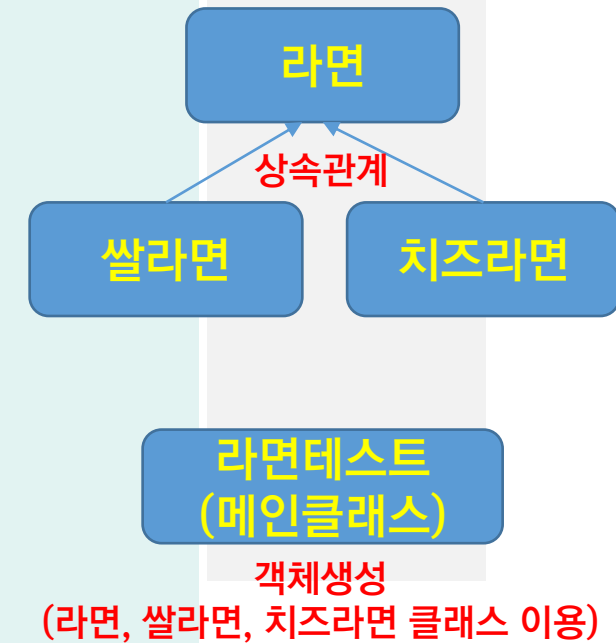
상속(Inheritance)

예제 10.1

CookingRamyon1.java

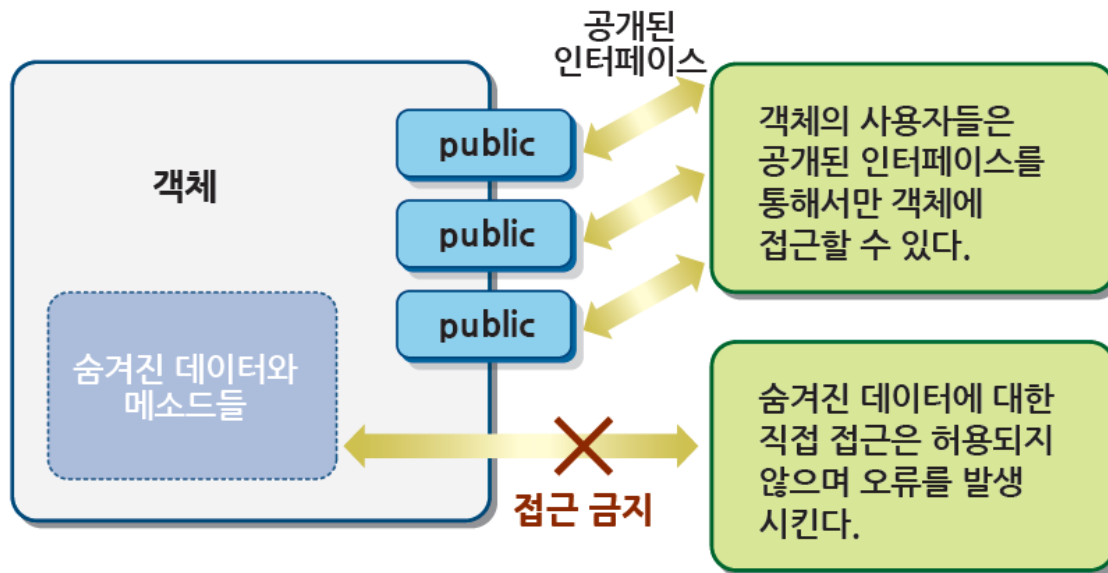
```
01: import java.util.Scanner;
02: class Ramyon {
03:     String ramyon="라면";
04:     String water="물";
05:     String onion="파";
06:     void boilwater() {
07:         System.out.println(water+"을 끓인다");
08:     }
09:     void cooking() {
10:         System.out.println(ramyon+"과 "+onion+"를 넣고 끓여 요리완성. 맛있게 드세요!!!");
11:     }
12: }
13:
14: class RiceRamyon extends Ramyon {
15:     String rice="떡";
16:     void topping() {
17:         System.out.println(rice+"을 넣는다");
18:     }
19: }
20:
21: class CheeseRamyon extends Ramyon {
22:     String cheese = "치즈";
23:     void topping() {
24:         System.out.println(cheese+"를 넣는다");
25:     }
26: }
27:
28: public class RamyonTest {
29:     public static void main(String args[]) {
30:         int s;
```

```
31:         do {
32:             System.out.print("만들고 싶은 요리를 입력하세요(1:일반라면,2:떡라면,3:치즈라면,99:종료) : ");
33:             Scanner stdin = new Scanner(System.in);
34:             s = stdin.nextInt();
35:             if (s == 1) {
36:                 Ramyon r = new Ramyon();
37:                 r.boilwater();
38:                 r.cooking();
39:             } else if (s == 2) {
40:                 RiceRamyon rr = new RiceRamyon();
41:                 rr.boilwater();
42:                 rr.topping();
43:                 rr.cooking();
44:             } else if (s == 3) {
45:                 CheeseRamyon cr = new CheeseRamyon();
46:                 cr.boilwater();
47:                 cr.topping();
48:                 cr.cooking();
49:             }
50:         } while (s != 99);
51:         System.out.print("감사합니다");
52:     }
53: }
```



캡슐화(Encapsulation)

- **캡슐화**란? 공개된 인터페이스를 통해서만 데이터 접근 가능
 - 객체에 포함된 정보의 손상과 오용 방지
 - 데이터가 바뀌어도 다른 객체에 영향을 주지 않으므로 독립성 유지



```
01 class MultiDiv extends PlusMinus {
02     int multi; ← 속성을 공개(한정자를 지정하지 않았으므로) 모드로 지정
03     private double div; ← 속성을 숨김 모드로 지정
04     public String multi(int x, int y) {
05         multi = x * y;
06         return "두 수의 곱은 " + multi;
07     }
08     public String div(int x, int y) {
09         div = (double) x / y;
10         return "두 수의 나눈 값은 " + div;
11     }
12 }
13 public class FourRulesTest1 {
14     public static void main(String args[]) {
15         String plus, minus, multi, div;
16         MultiDiv ob1 = new MultiDiv();
17         .....
18         ob1.multi = 200; ← 공개된 속성이므로 속성값을 직접 조정 가능
19         ob1.div = 3.6666; ← 숨긴 속성에 접근하므로 오류가 발생
20         .....
```


메시지(Message)

■ 메시지란? 객체에 일을 시키는 행위

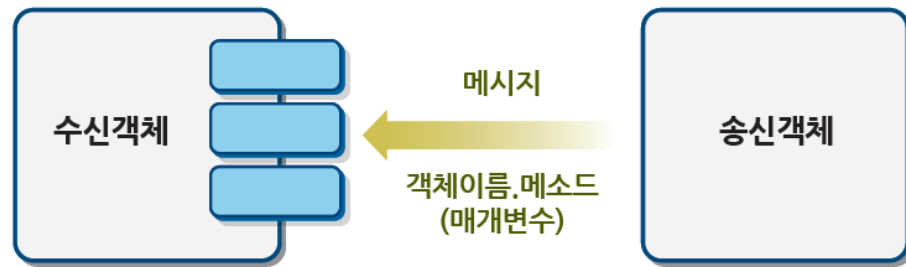


그림 7-13 객체 사이의 메시지 전달

```
01 public class FourRulesTest1 {
02     public static void main(String args[]) {
03         String plus, minus, multi, div;
04         MultiDiv ob1 = new MultiDiv();
05         plus = ob1.plus(50,30);
06         minus = ob1.minus(50,30);
07         multi = ob1.multi(50,30);
08         div = ob1.div(50,30);
09         .....
10     }
11 }
```

ob1 객체를 통하여 수행할 수 있는 메소드를 호출하는 메시지. 메시지의 구성은 객체 이름.메소드 이름(매개 변수)으로 구성된다.

추상화(Abstraction)

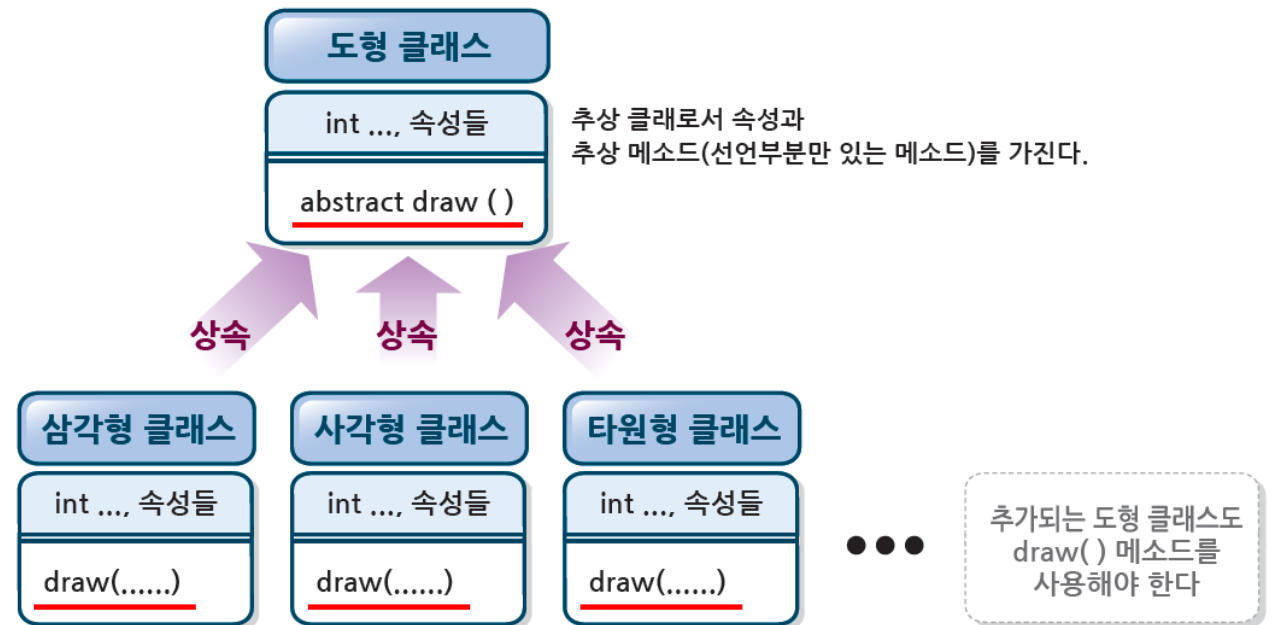
- **추상화**란? 복잡한 문제들 중에서 **공통적인 부분을 추출하여 추상클래스로 제공, 상속을 이용하여 추상클래스의 기능을 하위 클래스로 제공하는 방법**

추상화의 단계

- 1단계 : 현실 세계의 문제들이 가지는 공통적인 속성을 추출
- 2단계 : 공통 속성을 가지는 추상 클래스 작성
- 3단계 : 추상 클래스의 하위 클래스로 현실 세계의 문제들을 구현



그림 7-15 클래스를 연관시키지 않고 작성



추상 클래스의 하위 클래스는 추상 클래스에 선언된 추상 메소드를 치환하여 사용해야 한다.

그림 7-16 추상 클래스를 사용하여 체계적으로 클래스를 구성

추상화(따라해보기)

예제 11.6

AbstractTest1.java

```
01: abstract class Shape {
02:     abstract void draw();
03:     abstract void computeArea(double a, double b);
04: }
05: class Circle1 extends Shape {
06:     void draw() {
07:         System.out.println("원을 그리는 기능");
08:     }
09:     void computeArea(double r1, double r2) {
10:         System.out.println("원의 넓이 : " + (3.14 * r1 * r2));
11:     }
12: }
13: class Rectangle1 extends Shape {
14:     void draw() {
15:         System.out.println("사각형을 그리는 기능");
16:     }
17:     void computeArea(double h, double v) {
```

추상 클래스에 추상 메소드 선언

추상 클래스로부터 상속받아 클래스 생성
추상 메소드 오버라이딩

```
18:         System.out.println("사각형의 넓이 : " + (h * v));
19:     }
20: }
21: class Triangle1 extends Shape {
22:     void draw() {
23:         System.out.println("삼각형을 그리는 기능");
24:     }
25:     void computeArea(double a, double h) {
26:         System.out.println("삼각형의 넓이 : " + (a * h / 2));
27:     }
28: }
29:
30: public class AbstractTest1 {
31:     public static void main(String args[]) {
32:         System.out.println("==추상 메소드를 이용한 다형성==");
33:         Shape s = new Circle1();
34:         s.draw();
35:         s.computeArea(5.0, 5.0);
36:         s = new Rectangle1();
37:         s.draw();
38:         s.computeArea(5.0, 10.0);
39:         s = new Triangle1();
40:         s.draw();
41:         s.computeArea(5.0, 10.0);
42:     }
43: }
```

추상 클래스 형의 객체 변수에
하위 클래스의 객체를 배정

오버라이딩된 메소드 수행

하나의 추상 메소드를 가진 추상 클래스 생성

다형성(Polymorphism)

- **다형성**이란? 서로 다른 객체가 동일한 메시지에 대하여 서로 다른 방법으로 응답
 - 상속과 추상 클래스를 이용하여 다양한 변신(메모리 공간 절약)

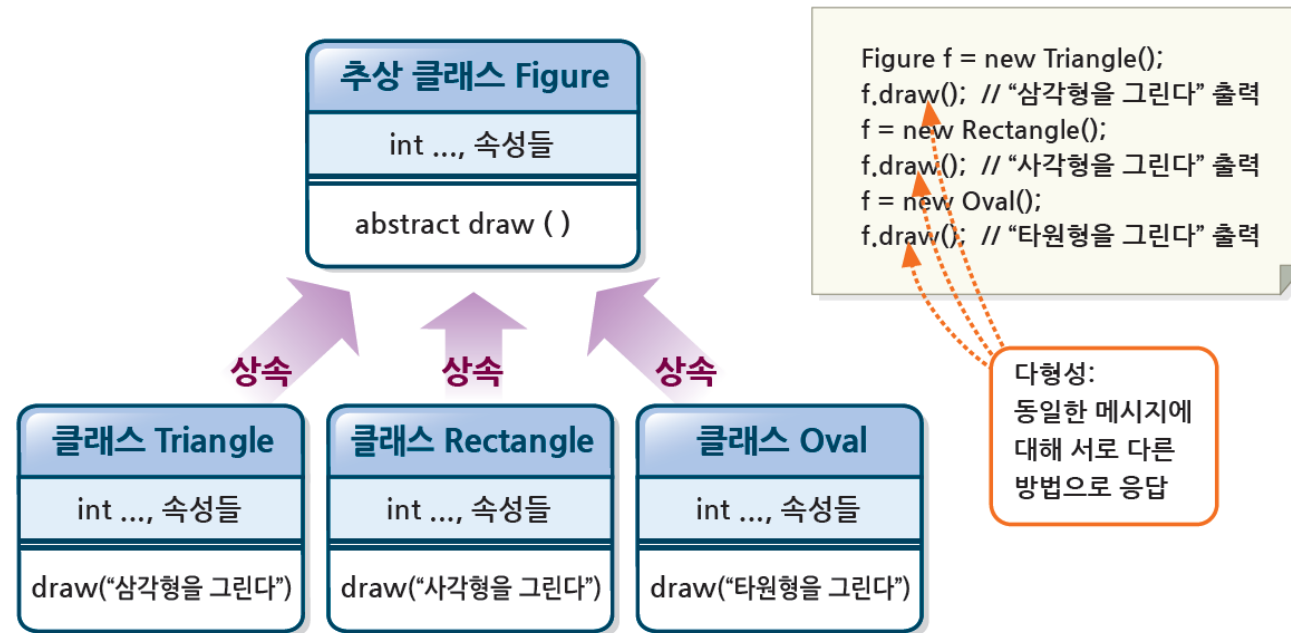


그림 7-17 다형성의 개념

다형성(따라해보기)

예제 11.4

PolymorphismTest1.java

```
01: class Am {
02:     int count=1; <----- count 값을 1로 설정
03:     void callme() {
04:         System.out.println("Am의 callme() 실행, count 값 : "+count);
05:     }
06: }
07: class Bm extends Am {
08:     int count=2; <----- count 값을 2로 설정
09:     void callme() { <----- 상위 클래스의 callme() 메소드 오버라이딩
10:         System.out.println("Bm의 callme() 실행, count 값 : "+count);
11:     }
12: }
13:
14: class Cm extends Am {
15:     int count=3; <----- count 값을 3으로 설정
16:     void callme() { <----- 상위 클래스의 callme() 메소드 오버라이딩
17:         System.out.println("Cm의 callme() 실행, count 값 : "+count);
18:     }
19: }
```

```
20:
21: class PolymorphismTest1 {
22:     public static void main(String args[]) {
23:         Am r = new Am(); <----- Am 클래스로부터 객체 생성력
24:         r.callme(); <----- callme() 호출. Am 클래스의 callme() 수행. count 값으로 1 출력문자열 출력
25:         System.out.println("r.count 값 : " + r.count); <-----
<----- r.count 출력. Am 클래스의 count 값 1이 출력 ----->
26:         r = new Bm(); <----- Am 클래스 형의 객체변수 r에 Bm 클래스의 객체 배정(형 변환)
27:         r.callme(); <----- callme() 호출. Bm 클래스의 callme() 수행. count 값으로 2 출력
28:         System.out.println("r.count 값 : " + r.count); <-----
<----- r.count 출력. Am 클래스의 count 값 1이 출력 ----->
29:         r = new Cm(); <----- Am 클래스 형의 객체변수 r에 Cm 클래스의 객체 배정(형 변환)
30:         r.callme(); <----- callme() 호출. Cm 클래스의 callme() 수행. count 값으로 3 출력
31:         System.out.println("r.count 값 : " + r.count); <-----
<----- r.count 출력. Am 클래스의 count 값 1이 출력 ----->
32:     }
33: }
```

클래스(속성)

클래스 선언 및 구성

■ 클래스 선언 : [public/final/abstract] class Class_name { ... }

- public : 모든 클래스에서 접근 가능한 클래스, 메인클래스에만 선언 가능
- 아무것도 작성안함 : 같은 패키지에 있는 클래스만 접근 가능
- final : 서브 클래스를 가질 수 없는 클래스(상속안됨, 종단클래스)
- abstract : 객체를 생성할 수 없는 클래스(추상클래스)

【형식】 클래스 선언

[public/final/abstract] class Class-name {

■ 클래스 구성 : 멤버변수, 생성자, 메소드로 구성(항상 3가지 요소를 갖진 않음)

- 그 중 클래스의 속성에 해당하는 구성요소는 멤버변수



클래스(속성-멤버변수)

- 클래스 구성요소 중 **멤버변수**는 객체변수, 클래스변수, 종단변수로 구분
 - 멤버변수는 접근 한정자 제공(public, private, protected)

[형식] 멤버 변수의 선언

[public/private/protected] [static] [final] 변수형 변수명;

- 생성자나 메소드 변수**는 **매개변수와 지역변수**로 구분

[형식] 생성자, 메소드에서의 매개 변수와 지역 변수의 선언

[final] 변수형 변수명;

[예]

```
public void cc(final int x, final int y) {  
    final int Max = 10;  
    String name="C.S.Kim";  
    private int num;  
}
```

←----- 메소드의 매개 변수에 final을 지정
←----- 메소드의 지역 변수로 final을 선언
←----- 지역 변수로 문자열 변수 선언
←----- 오류 발생. 객체 변수에만 사용 가능

클래스(속성-멤버변수)

예제 8.3

Box1Test1.java

1. 클래스설계

```
01: class Box1 {  
02:     int width;  
03:     int height;  
04:     int depth;  
05: }
```

3개의 속성을 가진 클래스 Box1을 선언

2. 객체 생성

```
06: public class Box1Test1 {  
07:     public static void main(String args[]) {  
08:         Box1 mybox1 = new Box1();  
09:         Box1 mybox2 = new Box1();  
10:         int vol1, vol2;  
11:     }
```

클래스 Box1으로부터 두 개의

3. 메시지(멤버변수 값 저장)

```
12:     mybox1.width = 78;  
13:     mybox1.height = 145;  
14:     mybox1.depth = 87;  
15:  
16:     mybox2.width = 48;  
17:     mybox2.height = 45;  
18:     mybox2.depth = 137;  
19:  
20:     vol1 = mybox1.width * mybox1.height * mybox1.depth;  
21:     System.out.println("첫 번째 박스의 부피는 " + vol1 + "입니다");  
22:  
23:     vol2 = mybox2.width * mybox2.height * mybox2.depth;  
24:     System.out.println("두 번째 박스의 부피는 " + vol2 + "입니다");  
25: }  
26: }
```

각 객체의 속성값을 직접 지정

박스의 부피를 계산하여 출력

클래스(속성-클래스변수)

■ 클래스변수? 멤버변수는 독립적, 클래스변수는 모든 객체가 공유

예제 8.6

Box3Test1.java

```
01: class Box3 {  
02:     int width;  
03:     int height;  
04:     int depth;  
05:     long idNum;  
06:     static long boxID = 0; ← 클래스 변수로 box  
07:     public Box3() { ←  
08:         idNum = ++boxID; ← 생성자에서 클래스  
09:     } ←  
10: }
```

```
11: class Box3Test1 {  
12:     public static void main(String args[]) {  
13:         Box3 mybox1 = new Box3(); ←  
14:         Box3 mybox2 = new Box3(); ← 객체를 생성. 생성자 수행  
15:         Box3 mybox3 = new Box3();  
16:         Box3 mybox4 = new Box3(); ←  
17:         ← 객체의 idNum이 증가되면서 출력  
18:         System.out.println("mybox1의 id 번호 : " + mybox1.idNum); ←  
19:         System.out.println("mybox2의 id 번호 : " + mybox2.idNum); ←  
20:         System.out.println("mybox3의 id 번호 : " + mybox3.idNum); ←  
21:         System.out.println("mybox4의 id 번호 : " + mybox4.idNum); ←  
22:         System.out.println("마지막 생성된 박스 번호는 " + Box3.boxID + "번  
    입니다."); ← 클래스명을 통하여 클래스 변수값 출력  
23:     } ← 객체명을 통하여 클래스 변수값 출력  
24: }
```

클래스(기능)

클래스(기능-생성자)

- 클래스에서 기능에 해당하는 구성요소는 **생성자**와 **메소드**
 - 생성자?** 객체 생성 시 **초기화**, 객체 생성 시 **자동 호출**, 클래스명과 동일, 반환 값 없음

예제 9.1 Box4Test1.java

```
01: class Box4 {
02:     int width;
03:     int height;
04:     int depth;
05:     public Box4(int w, int h, int d)
06:     {
07:         width = w;
08:         height = h;
09:         depth = d;
10:     }
11: }
12: public class Box4Test1 {
13:     public static void main(String args[]) {
14:         Box4 mybox1 = new Box4(10,20,30);
15:         //Box4 mybox2 = new Box4();
16:         int vol = mybox1.width * mybox1.height * mybox1.depth;
17:         System.out.println("박스의 부피 : " + vol);
18:     }
19: }
```

3개의 매개 변수를 가진 생성자 선언

속성에 값을 설정하는 초기화 과정

명시적 생성자를 사용하여 객체를 생성

오류 발생 적합한 생성자가 없다.

박스의 부피를 구한다.

클래스(기능-메소드)

- 클래스에서 기능에 해당하는 구성요소는 **생성자**와 **메소드**
 - **메소드?** 클래스의 핵심 구성요소로 기능을 나타냄
 - 접근 한정자는 멤버변수와 동일하게 public, private, protected 사용

【 형식 】 메소드

[public/private/protected][static/final/abstract/synchronized]

반환값형 메소드 이름([매개 변수들])

{

.....

지역 변수 및 메소드의 행위 기술

.....

}

static : 클래스 메소드

final : 종단 메소드

abstract : 추상 메소드

synchronized : 동기화 메소드

클래스(기능-생성자/메소드)

예제 9.5

Box8Test1.java

```
01: class Box8 {  
02:     int width;  
03:     int height;  
04:     int depth;  
05:     public Box8(int width, int height, int depth)  
06:     {  
07:         this.width = width;  
08:         this.height = height;  
09:         this.depth = depth;  
10:     }  
11:     int volume() ←  
12:     {  
13:         int vol = width * height * depth;  
14:         return vol;  
15:     } ←  
16: }
```

메소드 volume()에서 부피를
계산하여 결과를 반환

```
17: public class Box8Test1 {  
18:     public static void main(String args[]) {  
19:         Box8 mybox1 = new Box8(10,20,30);  
20:         //mybox1.width = 20; ← 접근 한정자를 사용하지 않았으므로, 값을 변경할 수 있다.  
21:         int vol1 = mybox1.volume(); ← 메소드를 호출하여 부피를 구한다.  
22:         System.out.println("정수 박스의 부피 : " + vol1);  
23:     }  
24: }
```

클래스(기능-클래스 메소드)

예제 9.7

Box10Test1.java

```
01: class Box10 {
02:     private int width;
03:     private int height;
04:     private int depth;
05:     private int vol;
06:     private long idNum;
07:     private static long boxID = 0;
08:     public Box10(int width, int height, int depth)
09:     {
10:         this.width = width;
11:         this.height = height;
12:         this.depth = depth;
13:         idNum = ++boxID;
14:         volume();
15:     }
16:     private void volume()
17:     {
```

객체의 모든 속성들을 private로 선언

클래스 변수도 private로 선언

생성자에서 idNum에 고유 번호를 부여

```
18:         vol = width * height * depth;
19:     }
20:     public String getvolume() {
21:         return idNum + "번 박스의 부피 : " + vol;
22:     }
23:     public static long getCurrentID() {
24:         // return idNum;
25:         return boxID;
26:     }
27: }
28: public class Box10Test1 {
29:     public static void main(String args[]) {
30:         Box10 mybox1;
31:         for(int i=1 ; i <= 5 ; i++) {
32:             mybox1 = new Box10(i,i+1,i+2);
33:             System.out.println(mybox1.getvolume());
34:         }
35:         System.out.println("마지막 생성된 박스 번호는 " + Box10.getCurrentID()
36:             + "번입니다");
37:         // System.out.println(Box10.boxID);
38:     }
```

박스의 고유 번호와 부피를 반환

현재의 박스 번호를 반환하는 클래스 메소드

idNum을 사용하면 오류 발생

클래스 변수 boxID 반환

클래스 메소드를 이용하여 현재의 박스 번호 출력

클래스 변수를 private로 선언함으로써 오류 발생

접근 한정자

접근 한정자(개념)

- 자바에서 **한정자**를 사용할 수 있는 대상? **클래스**, **멤버변수**, **메소드**
- 한정자에 따른 접근 가능 여부
 - **public** : 모든 클래스에서 사용 가능
 - **protected** : 동일 패키지 모두 접근 가능, 다른 패키지면 상속 관계만 허용
 - **default** : 동일 패키지만 접근 가능
 - **private** : 해당 클래스에서만 접근 가능

표 10-1 한정자에 따른 접근 가능 여부

접근 한정자	동일한 클래스	같은 패키지 내의 모든 클래스 (상속 여부에 상관없음)	다른 패키지	
			상속된 클래스	상속되지 않은 클래스
public	O	O	O	O
protected	O	O	O	X
default(한정자 없음)	O	O	X	X
private	O	X	X	X

접근 한정자(따라해보기)

예제 10.2

InheritanceTest1.java

```
01: class A {
02:     public int aa = 1;
03: }
04: class B extends A {
05:     private int bb = 2;
06:     public int bb() {
07:         return bb;
08:     }
09: }
10: class C extends B {
11:     int cc = 3;
12: }
13: public class InheritanceTest1 {
14:     public static void main(String[] args) {
15:         C objc = new C();
16:         System.out.println("objc객체의 객체 속성 변수 aa의 값은 " + objc.aa);
17:         System.out.println("objc객체의 객체 속성 변수 bb의 값은 " + objc.bb());
18:         //objc.bb에 접근하면 오류 발생
19:         System.out.println("objc객체의 객체속성변수 cc의 값은 " + objc.cc);
20:     }
21: }
```

02: 멤버 변수가 public으로 선언

04: 클래스 A로부터 상속

05: 멤버 변수가 private으로 선언

06: 메소드 bb() 선언. private 변수의 값을 반환

10: 클래스 B로부터 상속

11: 한정자 없이 멤버 변수 선언

private 속성에 직접 접근할 수 없으므로 메소드를 통해 값을 출력

객체의 public 속성에 직접 접근 가능

클래스 C로부터 객체 생성

한정자 없이 사용한 속성에 접근

예제 10.3

InheritanceTest2.java

```
01: class B1 {
02:     public int x = 500;
03:     public int y = 1000;
04: }
05: class B2 extends B1 {
06:     public String x = "처음 시작하는 자바";
07: }
08: public class InheritanceTest2 {
09:     public static void main(String args[]) {
10:         B2 b2 = new B2();
11:         System.out.println("객체 b2에 들어 있는 x,y 값 : " + b2.x + b2.y);
12:         B1 b1 = new B1();
13:         System.out.println("객체 b1에 들어 있는 x,y 값 : " + b1.x + b1.y);
14:     }
15: }
```

public 멤버 변수 선언

상위 클래스와 같은 이름으로 public x 선언. 상위 클래스의 값은 가려짐

객체 b2에 들어 있는 x,y 값 : " + b2.x + b2.y);

하위 클래스의 x 값 출력. y값은 상속된 값 출력

객체 b1에 들어 있는 x,y 값 : " + b1.x + b1.y);

상위 클래스의 속성값 xy 출력

오버로딩과 오버라이딩

생성자 오버로딩(1/2)

- **생성자 오버로딩**이란? **하나의 클래스에 여러 개의 생성자가 있는 경우(중첩)**
 - 생성자 오버로딩의 규칙 : 생성자 **매개 변수의 타입과 개수가 달라야 함**

예제 9.2

Box5Test1.java

```
01: class Box5 {
02:     int width;
03:     int height;
04:     int depth;
05:     public Box5()
06:     {
07:         width = 1;
08:         height = 1;
09:         depth = 1;
10:     }
11:     public Box5(int w)
12:     {
13:         width = w;
14:         height = 1;
15:         depth = 1;
16:     }
17:     public Box5(int w, int h)
18:     {
19:         width = w;
20:         height = h;
21:         depth = 1;
22:     }
23:     public Box5(int w, int h, int d)
24:     {
25:         width = w;
26:         height = h;
27:         depth = d;
28:     }
29: }
```

생성자 오버로딩 매개 변수의 형과 개수가 다르다.

생성자 오버로딩(2/2)

```
30: public class Box5Test1 {
31:     public static void main(String args[]) {
32:         Box5 mybox1 = new Box5(); ← 매개 변수가 없는 생성자 호출
33:         int vol = mybox1.width * mybox1.height * mybox1.depth;
34:         System.out.println("박스의 부피(매개 변수 없음) : " + vol);
35:         mybox1 = new Box5(10); ← 매개 변수가 하나 있는 생성자 호출
36:         vol = mybox1.width * mybox1.height * mybox1.depth;
37:         System.out.println("박스의 부피(매개 변수 1개) : " + vol);
38:         mybox1 = new Box5(10,20); ← 매개 변수가 두 개 있는 생성자 호출
39:         vol = mybox1.width * mybox1.height * mybox1.depth;
40:         System.out.println("박스의 부피(매개 변수 2개) : " + vol);
41:         mybox1 = new Box5(10,20,30); ← 매개 변수가 세 개 있는 생성자 호출
42:         vol = mybox1.width * mybox1.height * mybox1.depth;
43:         System.out.println("박스의 부피(매개 변수 3개) : " + vol);
44:     }
45: }
```

메소드 오버로딩

예제 9.10

Box1Test1.java

```
01: class Box12 {
02:     private int ivol;
03:     public Box12(int w, int h, int d)
04:     {
05:         ivol = w * h * d;
06:     }
07:     public String get_ivol() {
08:         return "박스의 부피는 " + ivol;
09:     }
10: }
11:
12: public class Box12Test1 {
13:     public static void main(String args[]) {
14:         Box12 mybox[] = new Box12[10]; // 10개의 요소를 가진 Box12 형의 배열 선언과 생성
15:         for(int i = 0 ; i < mybox.length ; i++) {
16:             mybox[i] = new Box12(i*10, i*20, i*30); // 배열의 각 요소에 Box12형의 객체를 생성
17:         }
18:         for(Box12 mb : mybox) {
19:             System.out.println(mb.get_ivol()); // 배열의 각 요소의 메소드를 호출하여 결과 출력
20:         }
21:     }
22: }
```

예제 9.11

OverloadTest1.java

```
01: class Overload{
02:     public void calc(){ // 오버로딩된 메소드
03:         System.out.println("매개 변수가 없습니다.");
04:     }
05:     public void calc(int width){
06:         System.out.println("정사각형의 넓이 : " + width * width);
07:     }
08:     public void calc(int width, int height){
09:         System.out.println("직사각형의 넓이 : " + width * height);
10:     }
11:     public void calc(int width, int height, int depth){
12:         System.out.println("직육면체의 부피 : " + width * height * depth);
13:     }
14: }
15:
16: public class OverloadTest1 {
17:     public static void main(String args[]){
18:         Overload ol = new Overload(); // 객체를 생성
19:         int input[] = new int[args.length]; // 입력한 매개 변수의 크기와 같은 정수 배열 생성
20:         for(int i=0; i<args.length; i++){
21:             input[i] = Integer.parseInt(args[i]); // 실행 시 입력한 문자열 배열의 요소를 정수로 바꾸어 정수 배열에 저장
22:         }
23:         switch (args.length){ // 정수 배열의 길이를 기준으로 switch문 수행
24:             case 0:
25:                 ol.calc();
26:                 break;
27:             case 1:
28:                 ol.calc(input[0]);
```

```
28:         break;
29:             case 2:
30:                 ol.calc(input[0], input[1]);
31:                 break;
32:             case 3:
33:                 ol.calc(input[0], input[1], input[2]);
34:                 break;
35:             default:
36:                 System.out.println("인수의 개수가 많습니다.");
37:         }
38:     }
39: }
```

배열의 길이에 따라 메소드 호출

calc()
calc(int width)
calc(int width, int height)
calc(int width, int height, int depth)

**동일한 이름의 메소드가 여러 개,
반드시 매개변수 개수 또는 자료형
이 달라야 함! (오버로딩)**

메소드 오버라이딩

- **오버라이딩**이란? 상속 관계에 있는 클래스들 간에 같은 이름의 메소드 정의
- **오버로딩이 중첩**(같은 메소드 여러 개)이라면, **오버라이딩은 치환**(상위 클래스의 메소드를 하위 클래스의 메소드로 교체)-**매개변수 개수, 타입 반드시 동일해야 함**

예제 10.7

OverridingTest1.java

```
01: class OIa {
02:     public void show(String str) { ← 하나의 매개 변수를 가진 show() 메소드 선언
03:         System.out.println("상위 클래스의 메소드 show(String str)수행 " + str);
04:     }
05: }
06: class OIb extends OIa {
07:     public void show() { ← 상속된 클래스에서 매개 변수 없는 show() 메소드 선언
08:         System.out.println("하위 클래스의 메소드 show() 수행");
09:     }
10: }
11: public class OverridingTest1 {
12:     public static void main(String args[]) {
13:         OIb oib = new OIb();
14:         oib.show("처음 시작하는 자바"); ←
15:         oib.show(); ← 각각에 적합한 메소드가 호출된다.
16:     }
17: }
```

예제 10.8

OverridingTest2.java

```
01: class OIaa {
02:     public void show(String str) {
03:         System.out.println("상위 클래스의 메소드 수행 : " + str);
04:     }
05: }
06: class OIbb extends OIaa {
07:     public void show(String s) { ← 상위 클래스의 show() 메소드를 그대로 다시 정의
08:         System.out.println("하위 클래스의 메소드 show() 수행 : " + s);
09:     }
10: }
11: public class OverridingTest2 {
12:     public static void main(String args[]) {
13:         OIbb oibb = new OIbb();
14:         oibb.show("쉽다 자바프로그래밍!!!"); ← OIbb 클래스의 show() 메소드 수행
15:     }
16: }
```

인터페이스

인터페이스

- **인터페이스**란? 자바의 다중 상속을 가능하게 하는 클래스(완벽한 추상화)
 - 추상 클래스는 추상 메소드 외에 다른 멤버변수나 일반 메소드 구성 가능
 - 반면에, 기본적으로 **인터페이스는 추상 메소드와 상수만 가질 수 있음**
 - 1.8 이후부터, **일반메소드(default 사용)와 클래스메소드(static 사용) 허용**

【 형식 】 interface

```
interface 인터페이스이름 [extends 인터페이스이름,[인터페이스이름]...] {  
    .....상수 선언;  
    .....메소드 선언;  
}
```

↑ 예약어 interface로 선언. 인터페이스끼리 상속 관계를 구성할 수 있다.

【 예 】

```
01: interface Sleeper {  
02:     public long ONE_SECOND = 1000;  
03:     public long ONE_MINUTE = 60000;  
04:     public void wakeup();  
05: }
```

← 인터페이스의 선언

← 상수의 선언 반드시 초기화해야 한다.

← 메소드의 선언 끝에 ";"을 붙인다.

인터페이스 개념(1/2)

- **인터페이스**란? 자바를 다중 상속을 가능하게 하는 클래스(완벽한 추상화)
 - 추상 클래스는 추상 메소드 외에 다른 멤버변수나 일반 메소드 구성 가능
 - 반면에, 기본적으로 **인터페이스는 추상 메소드와 상수만 가질 수 있음**
 - 1.8 이후부터, **일반메소드(default 사용)와 클래스메소드(static 사용) 허용**

【 형식 】 interface

```
interface 인터페이스이름 [extends 인터페이스이름,[인터페이스이름]...] {  
    .....상수 선언;  
    .....메소드 선언;  
}
```

↑ 예약어 interface로 선언. 인터페이스끼리 상속 관계를 구성할 수 있다.

【 예 】

```
01: interface Sleeper {  
02:     public long ONE_SECOND = 1000;  
03:     public long ONE_MINUTE = 60000;  
04:     public void wakeup();  
05: }
```

← 인터페이스의 선언

← 상수의 선언 반드시 초기화해야 한다.

← 메소드의 선언 끝에 ";"을 붙인다.

인터페이스 개념(2/2)

- 클래스에서 인터페이스를 붙여 사용하려면? **Implements 키워드 사용**
- 인터페이스를 사용하는 클래스는 **인터페이스의 모든 메소드를 오버라이딩 해야 함**

【형식】 인터페이스 포함(사용)

```
class 클래스 이름 extends 상위 클래스 이름 implements 인터 페이스 이름  
[인터페이스 이름, .....] {
```

.... 멤버 변수 선언

.... 생성자

.... 메소드 선언

.... 인터페이스에 선언된 모든 메소드를 오버라이딩하여 구현하여야 한다.

```
}
```

```
01: interface Sleeper { ← 인터페이스 Sleeper 선언  
02:     public long ONE_SECOND = 1000;  
03:     public long ONE_MINUTE = 60000;  
04:     public void wakeup();  
05: }  
06: interface Worker { ← 인터페이스 Worker 선언  
07:     public long WORK_TIME = 8;  
08:     public void sleep();  
09: }  
10: public class Man implements Sleeper, Worker { ← 두 개의 인터페이스를 포함  
11:     public void wakeup() { ← 메소드 오버라이딩  
12:         System.out.println("빨리 일어나 !!");  
13:     }  
14:     public void sleep() { ← 메소드 오버라이딩  
15:         System.out.println("빨리 자 !!");  
16:         .....  
17:     }  
18: }
```

인터페이스(따라해보기)

```
예제 11.8 InterfaceTest1.java

01: import java.util.Random;
02: interface IStack {
03:     public void push(int item);
04:     public int pop();
05: }
06: class FixedStack implements IStack {
07:     private int stack[];
08:     private int tos;
09:     FixedStack(int size) {
10:         stack = new int[size];
11:         tos = -1;
12:     }
13:     public void push(int item) {
14:         if(tos==stack.length-1)
15:             System.out.println("스택이 꽉 찼음");
16:         else
17:             stack[++tos] = item;
18:     }
19:     public int pop() {
20:         if(tos < 0) {
21:             System.out.println("스택이 비었음");
22:             return 0;
23:         }
24:         else
25:             return stack[tos--];
26:     }
27: }
```

```
28: public class InterfaceTest1 {
29:     public static void main(String args[]) {
30:         Random r = new Random();
31:         FixedStack mystack1 = new FixedStack(10);
32:         for(int i=0 ; i<10 ; i++)
33:             mystack1.push(r.nextInt(10));
34:         System.out.println("스택 : mystack1");
35:         for(int i=0 ; i<10 ; i++)
36:             System.out.print(mystack1.pop() + " ");
37:     }
38: }
```



인터페이스(따라해보기)

예제 11.9

InterfaceTest2.java

```
01: abstract class Figure1 {
02:     abstract void draw();
03: }
04: interface Shape1 {
05:     public void computeArea(double a, double b);
06: }
07: class Triangle3 extends Figure1 implements Shape1 {
08:     void draw() {
09:         System.out.println("삼각형을 그리는 기능");
10:     }
11:     public void computeArea(double a, double h) {
12:         System.out.println("삼각형의 넓이 : " + (a * h / 2));
13:     }
14: }
15: class Rectangle3 extends Figure1 implements Shape1 {
16:     void draw() {
17:         System.out.println("사각형을 그리는 기능");
18:     }
19:     public void computeArea(double h, double v) {
20:         System.out.println("사각형의 넓이 : " + (h * v));
21:     }
22: }
```

추상 클래스 선언

인터페이스 선언

추상 클래스와 인터페이스를 포함하는 클래스 작성

메소드 오버라이딩

메소드 오버라이딩

```
23: class Oval3 extends Figure1 implements Shape1 {
24:     void draw() {
25:         System.out.println("원을 그리는 기능");
26:     }
27:     public void computeArea(double r1, double r2) {
28:         System.out.println("원의 넓이 : " + (3.14 * r1 * r2));
29:     }
30: }
31: class Polydraw1 {
32:     public void pdraw(Figure1 f) {
33:         f.draw();
34:     }
35:     public void pcomputeArea(Shape1 s, double a, double b) {
36:         s.computeArea(a, b);
37:     }
38: }
39: public class InterfaceTest2 {
40:     public static void main(String args[]) {
41:         Polydraw1 p = new Polydraw1();
42:         Figure1 fg1 = new Triangle3();
43:         Figure1 fg2 = new Rectangle3();
44:         Figure1 fg3 = new Oval3();
45:         Shape1 sp1 = new Triangle3();
46:         Shape1 sp2 = new Rectangle3();
47:         Shape1 sp3 = new Oval3();
```

추상클래스(abstract)와
인터페이스(interface)를
이용한 다형성 구현 예제

```
48:         p.pdraw(fg1);
49:         p.pcomputeArea(sp1, 4, 4);
50:         p.pdraw(fg2);
51:         p.pcomputeArea(sp2, 4, 4);
52:         p.pdraw(fg3);
53:         p.pcomputeArea(sp3, 4, 4);
54:     }
55: }
```

인터페이스 형의 객체 변수로 메소드 호출

인터페이스 형의 객체 변수로 메소드 호출

인터페이스 형의 객체 변수로 메소드 호출

인터페이스 형의 객체 변수에
클래스의 객체를 배정

02

간단한 콘솔 프로그램 구현

회원관리프로그램

문제 정의

- 프로그램 이름 : **회원관리프로그램**
- 프로그램 메뉴 :
회원가입,
회원검색,
목록출력,
종료
- 전체 저장 가능한 회원 수 : **10명**

[회원가입]

다음 메뉴 중 하나를 입력하세요 :

1. 회원가입
2. 회원검색
3. 목록출력
4. 종료

1

이름 : a

나이 : 20

전화번호 : 010-111-1111

이메일 : aaa@kkk.com

회원 가입 완료!!!

다음 메뉴 중 하나를 입력하세요 :

1. 회원가입
2. 회원검색
3. 목록출력
4. 종료

1

이름 : b

나이 : 30

전화번호 : 010-222-2222

이메일 : bbb@kkk.com

회원 가입 완료!!!

[회원검색]

다음 메뉴 중 하나를 입력하세요 :

1. 회원가입
2. 회원검색
3. 목록출력
4. 종료

2

검색할 멤버명은?

b

*** 검색 결과 ***

이름 : b, 나이 : 30, 전화번호 : 010-222-2222, 이메일 : bbb@kkk.com

[목록출력]

다음 메뉴 중 하나를 입력하세요 :

1. 회원가입
2. 회원검색
3. 목록출력
4. 종료

3

이름 : a, 나이 : 20, 전화번호 : 010-111-1111, 이메일 : aaa@kkk.com

이름 : b, 나이 : 30, 전화번호 : 010-222-2222, 이메일 : bbb@kkk.com

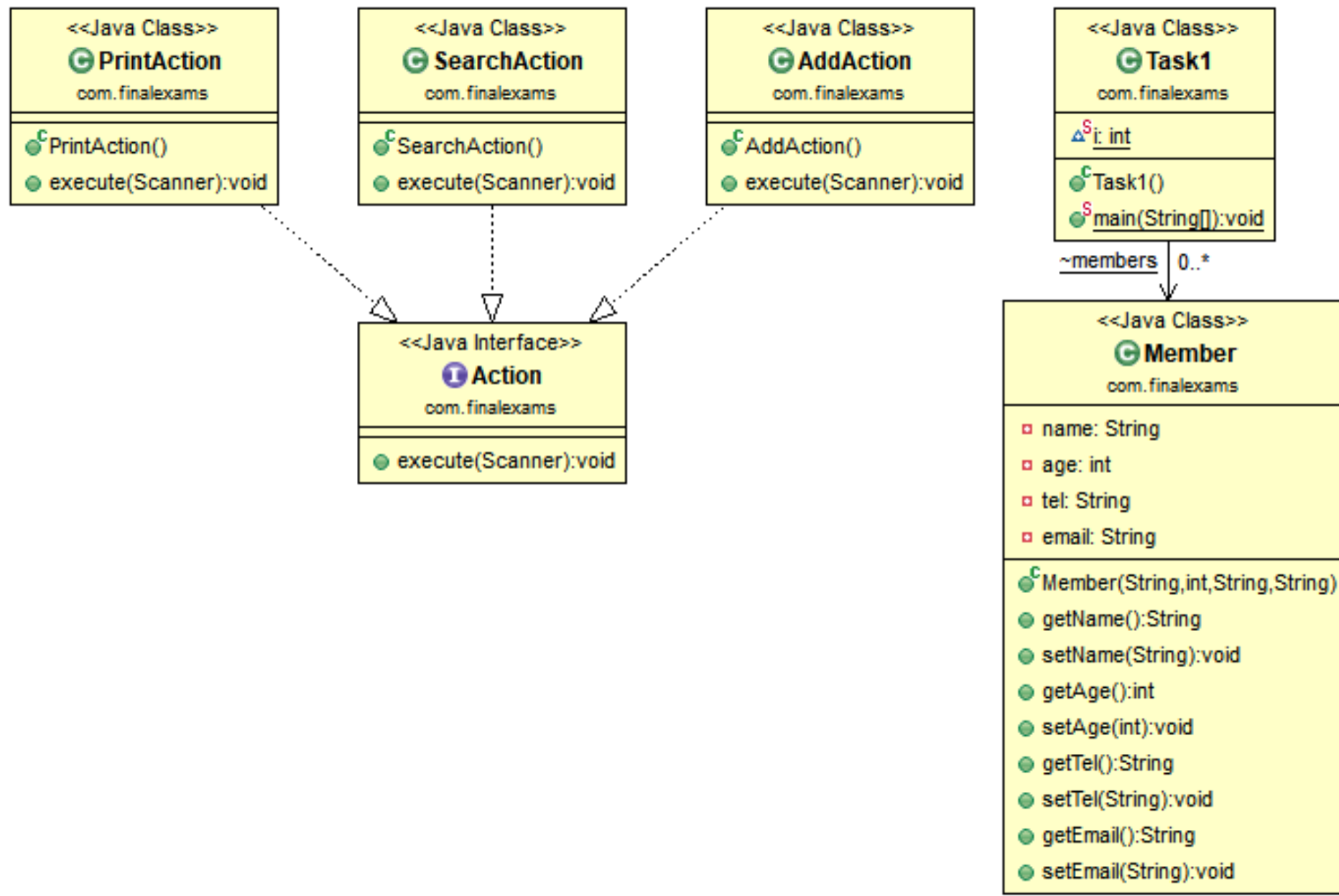
[종료]

다음 메뉴 중 하나를 입력하세요 :

1. 회원가입
2. 회원검색
3. 목록출력
4. 종료

4

프로그램 종료!



메인 클래스 코드 일부

```
public class Task1 {  
    static Member[] members = new Member[10]; //총 회원수는 10명으로 제한  
    static int i; // 객체 배열의 인덱스  
  
    public static void main(String[] args) {  
        ... // 데이터 입력을 위한 Scanner 클래스 객체 생성, isStop 변수 초기화(false)  
        do {  
            ... // 메뉴 출력, command 입력받기  
            switch(command) {  
                case "1":  
                    AddAction addAction = new AddAction();  
                    addAction.execute(sc);  
                    i++;  
                    break;  
                case "2":  
                    SearchActin searchAction = new SearchAction();  
                    searchAction.execute(sc);  
                    break;  
                ... // "3"과 "4"에 해당하는 case 문 처리하기  
            }  
        }while(!isStop);  
    }  
}
```

Quiz

과제물 제출

- 제출파일 : 6개 클래스 파일(Task1.java, Action.java 등)
- 제출처 : [LMS](#) - [\[과제\]](#) - [\[1주차\]](#)